

Common Part: All Questions

Common Part 1: PAL

Polynomial algorithms for standard graph problems. Combinatorial and number-theoretical algorithms, isomorphism, prime numbers. Search trees and their use. Text search based on finite automata.

Primary source repositories consulted: mohwald/oi-mszz, pan-sveta/oi-si-statnice, Pryx/oi-si-mszz, dralii/oi-mszz. The final text below is synthesized and corrected against standard algorithmic definitions.

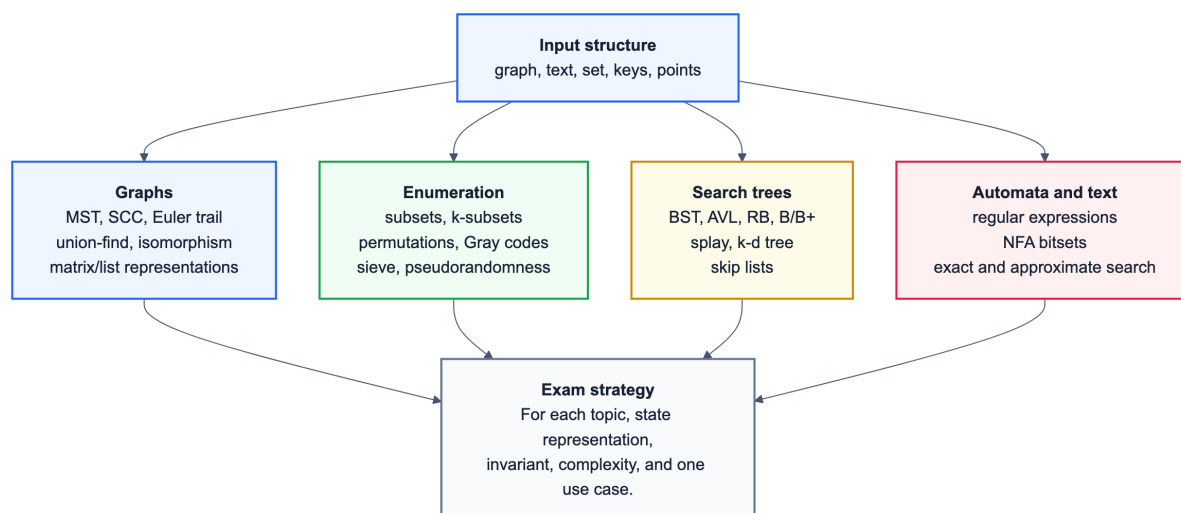


Figure 1: PAL topic map: graph algorithms, enumeration, search trees, and automata/text search.

PAL in Plain English

PAL is not one big idea. It is five small toolboxes. Learn what each toolbox is for, the main names, and one sentence you can say in the exam.

1. Recognize the topic.

Graph, generation, tree, or automaton?

2. Say the landmark.

Condition, algorithm name, or formula.

3. Finish with complexity.

Most PAL answers end with $O(\dots)$.

1

Graph basics

How to store a graph and talk about its size.

Know this

- **$n = \text{vertices}$, $m = \text{edges}$.**
- Adjacency list: good for sparse graphs, $O(n + m)$ memory.
- Adjacency matrix: $O(1)$ edge lookup, but $O(n^2)$ memory.

Watch out

- Distance matrix usually means shortest-path distances.
- Laplacian/incidence matrices are the algebraic view.

Exam sentence: For sparse graphs I would use adjacency lists because they take $O(n + m)$ memory; matrices are useful when I need $O(1)$ edge queries.

2

Graph algorithms

MST, components, Euler trails, and isomorphism.

Core algorithms

- **Prim/Kruskal/Boruvka** build minimum spanning trees.
- **Union-find** tracks components for Kruskal.
- **Kosaraju/Tarjan** find SCCs in $O(n + m)$.

Key conditions

- MST: cut property and cycle property.
- Euler undirected: all even, or exactly two odd.
- Euler directed: in-degree/out-degree balance.

Exam sentence: Kruskal sorts edges and adds an edge only if union-find says it connects two different components; sorting gives $O(m \log n)$.

3

Generation and numbers

Listing objects, finding primes, and simple pseudorandomness.

Counts

- Subsets: 2^n .
- k -subsets: $C(n, k)$.
- Permutations: $n!$.

Names to remember

- Gray code: consecutive bit strings differ in one bit.
- Sieve of Eratosthenes: all primes up to N .
- LCG: $X' = (aX + c) \bmod m$.

Exam sentence: The sieve marks multiples starting from p^2 and finds all primes up to N in $O(N \log \log N)$ time.

PAL in Plain English

Second half: search trees and automata/text search. These are mostly "choose the right structure" and "remember how the pattern is tracked."

4

Search trees

Different dictionaries for different workloads.

Balanced trees

- AVL: stricter balance, $O(\log n)$.
- Red-black: also $O(\log n)$, common in libraries.
- Splay: amortized $O(\log n)$, good with repeated access.

Special trees

- B/B+ trees: block storage and database indexes.
- k-d tree: geometric points in low dimensions.
- Skip list: randomized ordered dictionary.

Exam sentence: All balanced search trees aim for logarithmic height; the difference is how they maintain balance and what workload they are optimized for.

5

Automata and text search

A finite state machine remembers how much of the pattern has matched.

Automata

- DFA: exactly one next state.
- NFA: a set of possible states.
- Regex, DFA, and NFA describe regular languages.

Text search

- KMP: exact matching in $O(n + m)$.
- Levenshtein DP: approximate matching in $O(mn)$.
- Aho-Corasick: many patterns in one scan.

Exam sentence: KMP can be seen as an automaton over prefix lengths: after a mismatch, failure links say how much of the prefix is still usable.

How to study PAL in two days

First pass

Read these two pages and say each exam sentence out loud. You only need the rough map first.

Second pass

Go into the notes and attach details to each box: definitions, algorithms, and complexity.

Final pass

Cover the notes and answer from memory: "what is it, what algorithm, what complexity, what trap?"

Three common traps

- Do not mix directed and undirected Euler conditions.
- Graph invariants can disprove isomorphism, but usually do not prove it.
- $O(1)$ matrix edge lookup is bought with $O(n^2)$ memory.

1. Asymptotic Complexity and Graph Notation

Asymptotic notation

For non-negative functions $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$:

- $f(n) \in O(g(n))$ iff there exist constants $c > 0$ and n_0 such that $f(n) \leq cg(n)$ for all $n \geq n_0$.
- $f(n) \in \Omega(g(n))$ iff $g(n) \in O(f(n))$.
- $f(n) \in \Theta(g(n))$ iff $f(n) \in O(g(n))$ and $f(n) \in \Omega(g(n))$.
- $f(n) \in o(g(n))$ iff $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$.
- $f(n) \in \omega(g(n))$ iff $\lim_{n \rightarrow \infty} f(n)/g(n) = \infty$.

Typical growth order:

$$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$$

For graph algorithms, let $n = |V|$ and $m = |E|$.

Simple explanation

Asymptotic notation is a way to ignore machine speed and tiny implementation details. If one algorithm is $O(n)$ and another is $O(n^2)$, the first one scales much better: doubling the input roughly doubles its work, while the second one roughly quadruples it.

Basic graph terms

A graph is usually $G = (V, E)$. In an undirected graph, edges are unordered pairs $\{u, v\}$; in a directed graph, arcs are ordered pairs (u, v) .

- **Degree:** in an undirected graph, $\deg(v)$ is the number of incident edges. In directed graphs use $\deg^+(v)$ for out-degree and $\deg^-(v)$ for in-degree.
- **Walk:** sequence $v_0, e_1, v_1, \dots, e_k, v_k$ where each edge joins consecutive vertices.
- **Trail:** a walk with no repeated edge.
- **Path:** a walk with no repeated vertex.
- **Circuit / closed trail:** a trail that starts and ends in the same vertex.
- **Cycle:** a closed path, usually with at least one edge and no repeated vertices except the first/last.
- **Connected component:** maximal set of mutually reachable vertices in an undirected graph.
- **Strongly connected component:** maximal set of vertices in a directed graph where every vertex is reachable from every other.
- **Tree:** connected undirected graph with no cycles; equivalently, $|E| = |V| - 1$ and connected.

Graph representations

Adjacency matrix. Matrix $A \in \{0, 1\}^{n \times n}$, where $A_{ij} = 1$ iff edge (i, j) exists. For weighted graphs, store weights or ∞ for missing edges.

- Memory: $O(n^2)$.
- Edge existence query: $O(1)$.
- Iterating neighbors of one vertex: $O(n)$.
- Good for dense graphs.

Distance matrix. Matrix D , where D_{ij} stores the shortest-path distance from i to j , or ∞ if unreachable. It is not merely an input representation; it is often the output of all-pairs shortest paths.

Laplacian matrix. For an undirected graph,

$$L = \Delta - A,$$

where Δ is the diagonal matrix of degrees. L is central in spectral graph theory. The number of spanning trees can be obtained from any cofactor of L by Kirchhoff's matrix-tree theorem.

Incidence matrix. Matrix $B \in \{-1, 0, 1\}^{n \times m}$ for directed graphs: column $e = (u, v)$ has -1 at u , $+1$ at v , and zero elsewhere. For undirected graphs, entries are usually 1 at both endpoints.

Adjacency list. For each vertex, store a list of its outgoing neighbors.

- Memory: $O(n + m)$.
- Iterating all edges: $O(n + m)$.
- Edge existence query: $O(\deg(v))$, or expected $O(1)$ if neighbor sets are hashed.
- Best default for sparse graphs.

2. Minimum Spanning Trees, SCCs, Euler Trails, Union-Find, Isomorphism

Minimum spanning tree

Given connected undirected weighted graph $G = (V, E, w)$, a spanning tree is a connected acyclic subgraph containing all vertices. A minimum spanning tree (MST) has minimum total weight.

Important correctness principles:

- **Cut property:** for any cut, a lightest edge crossing the cut is safe for some MST.
- **Cycle property:** for any cycle, a heaviest edge on the cycle is not needed in some MST.

If edge weights are distinct, the MST is unique.

Prim-Jarnik algorithm

Prim grows one tree from an arbitrary start vertex.

1. Put the start vertex in the tree.
2. Repeatedly add the minimum-weight edge crossing from the tree to an outside vertex.
3. Stop when all vertices are included.

With an adjacency list and binary heap: $O(m \log n)$. With a Fibonacci heap: $O(m + n \log n)$.

With an adjacency matrix: $O(n^2)$, which is good for dense graphs.

Kruskal algorithm

Kruskal grows a forest.

1. Sort all edges by nondecreasing weight.
2. Scan edges in that order.
3. Add an edge iff it connects two different components.
4. Use union-find to maintain components.

Complexity: $O(m \log m)$ for sorting, i.e. $O(m \log n)$, plus almost-linear union-find overhead.

Example

Suppose edges sorted by weight are $(A, B) : 1$, $(B, C) : 2$, $(A, C) : 3$, $(C, D) : 4$. Kruskal first takes (A, B) , then (B, C) . It skips (A, C) because A and C are already connected, so adding it would create a cycle. Then it takes (C, D) . The MST edges are $(A, B), (B, C), (C, D)$.

Boruvka algorithm

Boruvka repeatedly connects every component using its cheapest outgoing edge.

1. Initially every vertex is one component.
2. For each component, find its lightest outgoing edge.
3. Add all such edges simultaneously and merge components.
4. Repeat until one component remains.

Each phase at least halves the number of components, so there are $O(\log n)$ phases. A simple implementation runs in $O(m \log n)$.

Union-find

Union-find maintains a partition of elements into disjoint sets.

- **make-set(x)**: create singleton set.
- **find(x)**: return representative of the set containing x .
- **union(x,y)**: merge the sets containing x and y .

Efficient implementation:

- Store each set as a rooted tree.
- Use **union by rank/size**: attach the shallower/smaller tree below the deeper/larger one.
- Use **path compression** during **find**: make visited nodes point directly to the root.

For q operations on n elements, complexity is $O((n+q)\alpha(n))$, where α is the inverse Ackermann function, effectively constant in practice.

Simple explanation

Union-find is useful when the only question is “are these two things already in the same group?” In Kruskal’s MST algorithm, each accepted edge merges two groups. Path compression makes future questions faster by making every visited vertex point almost directly to the group representative.

Strongly connected components

A strongly connected component (SCC) in a directed graph is a maximal subset $C \subseteq V$ such that for all $u, v \in C$, both $u \rightsquigarrow v$ and $v \rightsquigarrow u$.

The condensation graph of SCCs is always a DAG.

Kosaraju-Sharir algorithm

1. Run DFS on G and record vertices in decreasing finish time.
2. Reverse all edges to get G^R .
3. Process vertices in decreasing finish time; each DFS tree in G^R is one SCC.

Complexity: $O(n + m)$. It is simple, but needs the reversed graph and two DFS passes.

Tarjan algorithm Tarjan finds SCCs in one DFS using a stack.

For each vertex v maintain:

- **index[v]**: DFS discovery order.
- **lowlink[v]**: smallest discovery index reachable from v using zero or more tree edges plus at most one back edge to a vertex still on the stack.

When **lowlink[v] == index[v]**, v is the root of an SCC; pop stack vertices until v is popped.

Complexity: $O(n + m)$.

Euler trail

An Euler trail uses every edge exactly once. An Euler circuit is a closed Euler trail.

For an undirected graph, ignoring isolated vertices:

- Euler circuit exists iff every vertex has even degree and the non-isolated part is connected.
- Euler trail with different endpoints exists iff exactly two vertices have odd degree.

For a directed graph, ignoring isolated vertices and requiring all nonzero-degree vertices to lie in one weakly connected component:

- Euler circuit exists iff $\deg^+(v) = \deg^-(v)$ for every vertex.
- Euler trail from s to t exists iff $\deg^+(s) = \deg^-(s) + 1$, $\deg^-(t) = \deg^+(t) + 1$, and all other vertices have equal in/out degree.

Hierholzer algorithm: follow unused edges until a circuit/trail closes, then splice in additional circuits where unused edges remain. Runs in $O(n + m)$.

Exam tip

Do not mix the undirected and directed Euler conditions. Undirected graphs use even/odd degree. Directed graphs use equality of in-degree and out-degree, except possibly at the start and end vertices of an open trail.

Graph isomorphism

Graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are isomorphic if there is a bijection $f : V_1 \rightarrow V_2$ such that

$$\{u, v\} \in E_1 \iff \{f(u), f(v)\} \in E_2.$$

Necessary invariants include equal numbers of vertices and edges, degree multiset, connected-component sizes, cycle counts, spectrum of the adjacency/Laplacian matrix, etc. These invariants can quickly disprove isomorphism, but most are not complete: equal invariant values do not always imply isomorphism.

General graph isomorphism is in NP. It is not known to be in P nor NP-complete; the best-known general theoretical result is quasipolynomial time. Practical solvers use refinement, canonical labeling, and backtracking.

Tree isomorphism

Tree isomorphism is solvable in linear or near-linear time.

For unrooted trees:

1. Find the center(s) by repeatedly removing leaves, or via longest path midpoint.
2. Root each tree at its center. If a tree has two centers, try both rootings.
3. Compute a canonical code bottom-up: a leaf has code $()$, and an internal node has code consisting of sorted child codes inside parentheses.
4. Trees are isomorphic iff their canonical root codes match.

Sorting child codes naively gives $O(n \log n)$; careful integer relabeling by levels gives $O(n)$.

3. Combinatorial Generation, Prime Numbers, and Pseudorandom Numbers

Subsets

All subsets of an n -element set can be generated by bit masks from 0 to $2^n - 1$. Bit i tells whether element i is included.

- Count: 2^n .
- Time to output all subsets: $\Theta(n2^n)$ if each subset is printed explicitly.

Recursive include/exclude generation:

```
generate(i):  
    if i == n: output current subset  
    else:  
        exclude element i; generate(i+1)  
        include element i; generate(i+1)
```

k-element subsets

There are $\binom{n}{k}$ k -element subsets. Common generation methods:

- Recursive choice with pruning.
- Lexicographic next-combination algorithm.
- Revolving-door order, where consecutive combinations differ by one removed and one inserted element.

Lexicographic next-combination for indices $a_1 < \dots < a_k$:

1. Find largest i such that $a_i < n - k + i$.
2. Increment a_i .
3. Set $a_j = a_{j-1} + 1$ for all $j > i$.

Permutations

There are $n!$ permutations.

Useful algorithms:

- Lexicographic **next_permutation**: find longest non-increasing suffix, swap pivot with the next larger element in suffix, reverse suffix.
- Heap's algorithm: simple recursive generation with swaps.
- Johnson-Trotter: adjacent transpositions; consecutive permutations differ by swapping neighboring elements.

Gray codes

A binary Gray code lists all 2^n bit strings so that consecutive strings differ in exactly one bit. The reflected binary Gray code can be computed as

$$G(i) = i \oplus (i \gg 1).$$

Gray codes are useful when changing one decision at a time is cheaper than rebuilding a full object, e.g. iterating subsets while maintaining an aggregate value.

Example

For $n = 3$, reflected Gray code is 000, 001, 011, 010, 110, 111, 101, 100. Notice that each neighboring pair differs in one bit only. This is useful when each bit means “include this item in the subset” and you want to update the subset by one small change.

Prime numbers and sieve of Eratosthenes

A prime number is an integer $p > 1$ with only divisors 1 and p .

Trial division tests primality of n by trying divisors up to $\sqrt{n}$, so it takes $O(\sqrt{n})$ arithmetic divisions.

The sieve of Eratosthenes finds all primes $\leq N$:

1. Mark all numbers $2, \dots, N$ as potential primes.
2. For $p = 2, 3, \dots, \lfloor \sqrt{N} \rfloor$, if p is still marked, mark all multiples $p^2, p^2 + p, \dots$ composite.
3. Remaining marked numbers are primes.

Complexity: $O(N \log \log N)$ time and $O(N)$ memory.

For very large numbers, probabilistic primality tests such as Miller-Rabin are common; deterministic polynomial-time primality testing exists, but is less practical for ordinary use.

Pseudorandom numbers and LCG

A pseudorandom number generator (PRNG) is deterministic but should imitate randomness for intended statistical purposes.

Important properties:

- Long period.
- Uniform distribution over the output range.
- Low correlation between consecutive values.
- Reproducibility from a seed.
- For cryptography, unpredictability is also required; ordinary LCGs are not cryptographically secure.

A linear congruential generator (LCG) is

$$X_{n+1} = (aX_n + c) \bmod m.$$

The Hull-Dobell theorem gives full period m for a mixed LCG iff:

1. $\gcd(c, m) = 1$.
2. Every prime factor of m divides $a - 1$.
3. If $4 \mid m$, then $4 \mid a - 1$.

4. Search Trees

Binary search tree

A binary search tree (BST) stores keys with invariant:

$$\text{left subtree keys} < \text{node key} < \text{right subtree keys}.$$

Operations search/insert/delete take $O(h)$ where h is tree height. In a balanced tree $h = O(\log n)$; in the worst unbalanced case $h = O(n)$.

AVL tree

An AVL tree is a self-balancing BST where every node has balance factor

$$bf(v) = \text{height}(\text{left}(v)) - \text{height}(\text{right}(v)) \in \{-1, 0, 1\}.$$

After insertion/deletion, rotations restore balance:

- single right rotation,
- single left rotation,
- left-right rotation,
- right-left rotation.

Height is $O(\log n)$, so search, insert, and delete are $O(\log n)$.

Red-black tree

A red-black tree is a BST with color bits satisfying:

1. Every node is red or black.
2. The root is black.
3. Leaves/NIL sentinels are black.
4. A red node has black children.
5. Every path from a node to descendant NIL leaves has the same number of black nodes.

These rules imply height at most $2 \log_2(n+1)$. Search/insert/delete are $O(\log n)$. Red-black trees are often slightly less strictly balanced than AVL trees, but need fewer rotations in update-heavy workloads.

B-tree

A B-tree is a balanced multiway search tree optimized for block storage.

For minimum degree t :

- Each internal node except root has between $t - 1$ and $2t - 1$ keys.
- Each internal node with k keys has $k + 1$ children.
- All leaves are at the same depth.

Operations are $O(\log_t n)$ node visits. Since one node can match a disk/cache block, B-trees are used heavily in databases and filesystems.

B+ tree

A B+ tree stores all records in leaves; internal nodes contain only separator keys. Leaves are linked in sorted order.

Advantages over B-trees:

- Better range scans.
- Higher fanout in internal nodes.
- Stable leaf-level sequential access.

This is the standard index structure in many database systems.

Splay tree

A splay tree is a self-adjusting BST. After every access, insertion, or deletion helper access, the touched node is moved to the root by rotations:

- **zig**: parent is root.
- **zig-zig**: node and parent are both left children or both right children.
- **zig-zag**: node and parent are opposite-side children.

Worst-case single operation can be $O(n)$, but amortized complexity is $O(\log n)$. Splay trees exploit temporal locality: recently accessed keys become fast to access again.

k-d tree

A k-d tree stores points in k -dimensional space. At depth d , split by coordinate $d \bmod k$.

Construction:

1. Choose splitting coordinate.
2. Choose median point by that coordinate.
3. Recurse on points below/above the median.

Balanced construction costs $O(n \log n)$ or better with selection algorithms. Orthogonal range queries and nearest-neighbor queries are efficient in low dimensions, but performance deteriorates in high dimensions.

Nearest neighbor search in a k-d tree

1. Descend to the leaf where the query point would be inserted.
2. Keep current best point and distance.
3. Backtrack. At each split, first explore the side containing the query.
4. Explore the other side only if its bounding region could contain a closer point than the current best.

Average performance is often $O(\log n)$ in low dimensions, but worst case is $O(n)$.

Skip list

A skip list is a probabilistic ordered dictionary with multiple linked-list levels. Each element is promoted to the next level with probability p , commonly $1/2$.

Expected height is $O(\log n)$. Search, insert, and delete take expected $O(\log n)$ time and $O(n)$ expected space. Skip lists are simpler than balanced trees and work well in concurrent settings.

Complexity summary

Structure	Search	Insert	Delete	Notes
Unbalanced BST	$O(h)$, worst $O(n)$	$O(h)$	$O(h)$	Simple, input-order sensitive
AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$	Strict balance
Red-black	$O(\log n)$	$O(\log n)$	$O(\log n)$	Common library map/set
B-tree / B+ tree	$O(\log_t n)$	$O(\log_t n)$	$O(\log_t n)$	Block-oriented storage
Splay	amortized $O(\log n)$	amortized $O(\log n)$	amortized $O(\log n)$	Adapts to access locality
k-d tree	avg. $O(\log n)$	avg. $O(\log n)$	varies	Geometric data
Skip list	expected $O(\log n)$	expected $O(\log n)$	expected $O(\log n)$	Randomized

5. Finite Automata, Regular Languages, and Text Search

Finite automata

A deterministic finite automaton (DFA) is

$$A = (Q, \Sigma, \delta, q_0, F),$$

where Q is a finite state set, Σ alphabet, $\delta : Q \times \Sigma \rightarrow Q$ transition function, q_0 initial state, and $F \subseteq Q$ accepting states.

A nondeterministic finite automaton (NFA) allows multiple transitions and possibly ϵ -transitions:

$$\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q.$$

DFA and NFA recognize exactly the regular languages. Every NFA can be determinized by subset construction, possibly producing up to $2^{|Q|}$ DFA states.

Regular expressions and regular operations

Regular expressions denote regular languages. Basic constructs:

- $\emptyset$, ϵ , symbols $a \in \Sigma$.
- Union $R|S$.
- Concatenation RS .
- Kleene star R^* .

Regular languages are closed under union, intersection, complement, concatenation, Kleene star, difference, reversal, and homomorphism. Closure is usually proved by automaton constructions, e.g. product automata for intersection.

Bit representation of NFAs

For small/medium automata, the active NFA state set can be stored as a bit vector. One transition step becomes bit operations:

1. Maintain bitset S of active states.
2. For input character a , compute next active states using precomputed transition masks.
3. Apply epsilon closure if needed.

This is the basis of bit-parallel text searching. On a word-RAM with word size w , simulating m pattern states can cost $O(\lceil m/w \rceil)$ per text character.

Simple explanation

An NFA can be in many states at once. A bit vector is just a compact checklist of those active states. Instead of looping over states one by one, the computer can update many bits with one CPU operation.

Exact pattern matching

Given pattern P of length m and text T of length n .

Naive search: try every alignment and compare characters. Worst-case $O(nm)$.

Automaton/KMP view: build a DFA-like prefix automaton whose state is the length of the longest prefix of P matched so far. KMP computes failure links/prefix function in $O(m)$ and scans in $O(n)$, total $O(n + m)$.

Boyer-Moore family: compares from the end of the pattern and uses bad-character/good-suffix shifts. Worst-case variants can be linear; in practice it is often sublinear on natural text.

Shift-And / Shift-Or: bit-parallel exact matching. Precompute character masks for pattern positions. Each text character updates a bit vector; a match is found when the accepting bit is set. Complexity $O(n \lceil m/w \rceil)$.

Approximate pattern matching

Hamming distance between equal-length strings is the number of positions where they differ. Approximate matching with Hamming distance at most k allows substitutions only.

Levenshtein distance is edit distance with insertions, deletions, and substitutions. Dynamic programming recurrence:

$$D[i, j] = \min \begin{cases} D[i-1, j] + 1 & \text{delete} \\ D[i, j-1] + 1 & \text{insert} \\ D[i-1, j-1] + [P_i \neq T_j] & \text{substitute/match} \end{cases}$$

Full DP is $O(mn)$. If only distance $\leq k$ matters, compute a diagonal band of width $2k + 1$ for $O(kn)$. Bit-parallel algorithms such as Myers' algorithm can be much faster in practice for moderate pattern length.

Example

The Levenshtein distance between **cat** and **cut** is 1: substitute **a** by **u**. The distance between **cat** and **cart** is also 1: insert **r**. Hamming distance would not allow insertion, so it only applies to equal-length strings.

Approximate matching can also be modeled by an automaton whose states encode pattern position and number of errors used. For Levenshtein distance k , the automaton has roughly $O(mk)$ relevant states.

Dictionary automata

For a set of patterns, a trie recognizes prefixes of dictionary words. Aho-Corasick augments the trie with failure links, analogous to KMP failure links, so all dictionary occurrences in a text are found in one pass.

Construction:

1. Insert all patterns into a trie.
2. Build failure links by BFS.
3. Store output lists at terminal states and inherited failure states.
4. Scan text by transitions plus failure fallback.

Complexity:

- Construction: $O(\text{total pattern length} \cdot |\Sigma|)$ for full transition tables, or linear in stored transitions with maps.
- Search: $O(n + z)$, where z is the number of reported occurrences.

Exam checklist

Before answering, be able to:

- Define graph representations and compare their complexities.
- State MST algorithms, their invariants, and their time complexity.
- Distinguish undirected and directed Euler conditions.
- Explain union-find optimizations and the inverse Ackermann bound.
- Explain why tree isomorphism is easy but general graph isomorphism is special.
- Generate subsets, k -subsets, permutations, and Gray codes.
- Give sieve and LCG formulas and complexities.
- Compare AVL, red-black, B/B+, splay, k -d trees, and skip lists.
- Define DFA/NFA/regular expressions and connect automata to exact, approximate, and dictionary text search.

Common Part 2: TAL

Problem/language complexity classes with respect to the time complexity of their solution and memory complexity including undecidable problems/languages.

Primary source repositories consulted: mohwald/oi-mszz, pan-sveta/oi-si-statnice, Pryx/oi-si-mszz, dralii/oi-mszz. The final text below is synthesized and corrected against standard theory of computation.

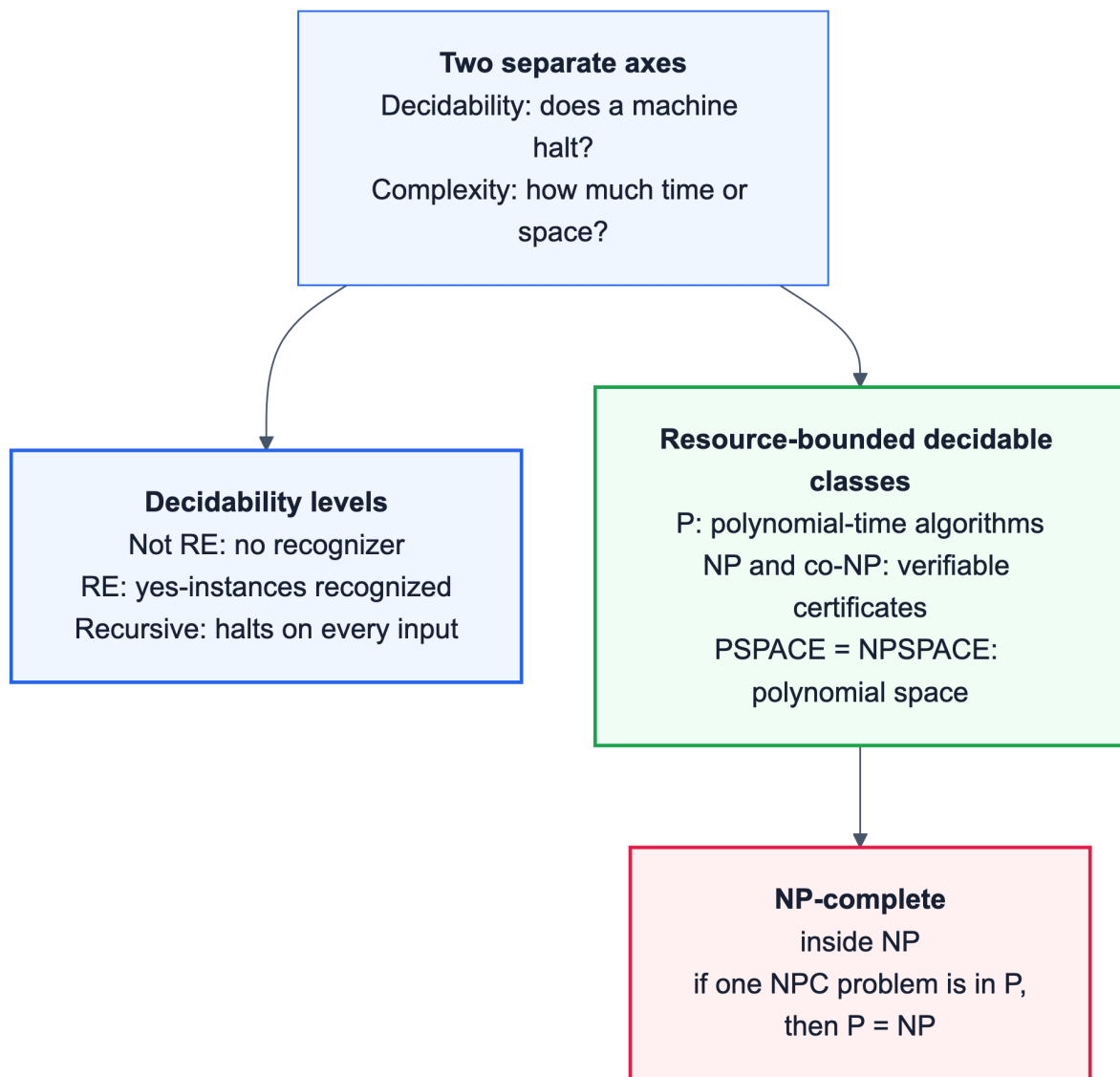


Figure 2: TAL hierarchy: resource-bounded classes sit inside decidable languages, while decidability classes form a separate axis.

TAL in Plain English

TAL is about sorting problems by what machines can solve, how fast they can solve them, how much memory they need, and which problems cannot be solved at all.

1. Identify the world.

Time, space, randomness, or decidability?

2. Name the class.

P, NP, PSPACE, RP, RE, decidable...

3. Give the reason.

Algorithm, certificate, reduction, or contradiction.

Complexity basics

What resources does an algorithm use?

- Time = number of steps.
- Space = working memory, usually not counting read-only input.
- Correctness = invariant for truth, variant for termination.

Turing machines

The formal model behind algorithms.

- Deterministic: one next move.
- Nondeterministic: many branches; accept if one branch accepts.
- Multitape changes speed polynomially, not the big classes.

P, NP, reductions

Can we solve it fast, or verify a solution fast?

- **P**: polynomial-time solving.
- **NP**: polynomial-time verification/certificate.
- **NP-hard**: everything in NP reduces to it.

Space and randomness

Memory-bounded and probability-bounded computation.

- **PSPACE**: polynomial memory.
- Savitch: $\text{NPSpace} = \text{PSPACE}$.
- RP/co-RP/ZPP differ by one-sided or zero error.

Undecidability

Some yes/no questions have no always-correct terminating algorithm.

- Decidable: halts with yes/no on every input.
- RE: halts on yes instances, may loop on no.
- Halting problem: classic contradiction by diagonalization.

How to answer

Most TAL answers are classification plus proof idea.

- Define the class/problem.
- State the inclusion or theorem.
- Use reduction, certificate, simulation, or diagonalization.

Exam sentence: NP means solutions can be verified in polynomial time; NP-complete problems are in NP and at least as hard as every NP problem by polynomial reduction.

Three traps

- NP does not mean "not polynomial"; P may still equal NP.
- Reduce from a known hard problem to prove your target hard.
- Semi-decidable is not the same as decidable.

Use this page first; then drill the formal definitions and theorem statements.

1. Asymptotic Growth, Complexity, and Correctness

Asymptotic growth

For non-negative functions f, g :

- $f \in O(g)$: f is eventually bounded above by a constant multiple of g .
- $f \in \Omega(g)$: f is eventually bounded below by a constant multiple of g .
- $f \in \Theta(g)$: both $O(g)$ and $\Omega(g)$.
- $f \in o(g)$: $f/g \rightarrow 0$.
- $f \in \omega(g)$: $f/g \rightarrow \infty$.

Asymptotic notation ignores constants and lower-order terms. It describes growth as input size n tends to infinity.

Time and space complexity

The **time complexity** of an algorithm is the number of elementary steps as a function of input size. The **space complexity** is the amount of working memory used as a function of input size.

For Turing machines:

- Time $T(n)$: maximum number of transitions on inputs of length n .
- Space $S(n)$: maximum number of work-tape cells used on inputs of length n .

Usually the read-only input tape is not counted toward space complexity.

Complexity of a **problem** is the complexity of the best possible algorithm in the chosen computational model. For robust classes such as P, NP, PSPACE, the exact reasonable model does not matter up to polynomial factors.

Correctness of algorithms

An algorithm is **partially correct** if, whenever it terminates, its output satisfies the specification. It is **totally correct** if it is partially correct and always terminates.

Loop invariant A loop invariant is a statement true:

1. before the loop starts,
2. before and after every iteration,
3. and strong enough that, together with loop termination, implies the postcondition.

Proof pattern:

- Initialization: invariant holds before first iteration.
- Maintenance: if it holds before an iteration, it holds after it.
- Termination: invariant plus negated loop guard implies correctness.

Loop variant A loop variant is a value from a well-founded ordered set, usually non-negative integers, that strictly decreases on every iteration. It proves termination because there is no infinite decreasing sequence.

2. Turing Machines

Deterministic Turing machine

A deterministic Turing machine (DTM) is commonly defined as

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{acc}, q_{rej}),$$

where:

- Q is a finite set of states.
- Σ is the input alphabet.
- $\Gamma \supseteq \Sigma$ is the tape alphabet, including blank symbol.
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$ is a partial transition function.
- q_0 is the start state.
- q_{acc} and q_{rej} are halting accept/reject states.

At each step the machine reads one tape cell, writes a symbol, moves the head, and changes state.

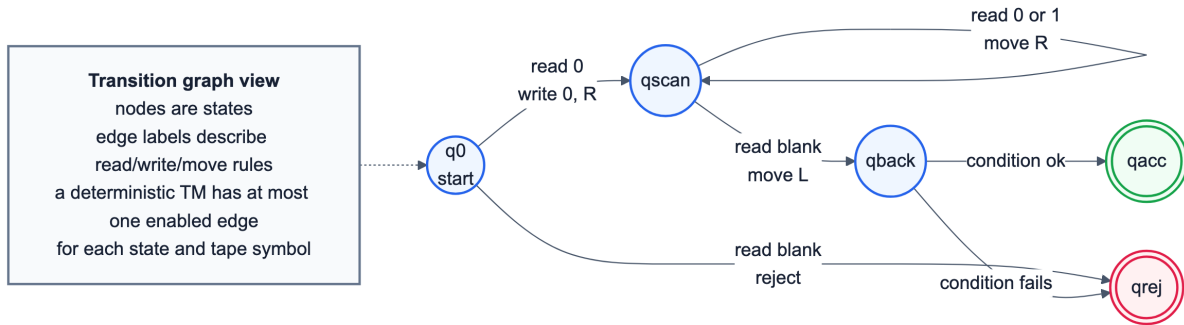


Figure 3: Turing-machine transition graph: states are nodes and transition rules are labeled directed edges.

A DTM **decides** a language if it halts on every input and accepts exactly strings in the language. It **recognizes** a language if it accepts exactly strings in the language, but may loop forever on strings outside the language.

Multitape Turing machine

A multitape TM has several tapes and heads. In one step it reads all head symbols, writes all head symbols, moves each head, and changes state.

Multitape machines are not more powerful than single-tape machines. A single-tape machine can simulate a k -tape machine by encoding all tapes on one tape. If the multitape machine runs in $T(n)$ time, a standard single-tape simulation runs in $O(T(n)^2)$ time. Thus the difference does not affect polynomial-time classes.

Nondeterministic Turing machine

A nondeterministic Turing machine (NTM) may have several possible transitions from one configuration. It accepts an input if at least one computation branch accepts.

Equivalent views:

- Branching computation tree.

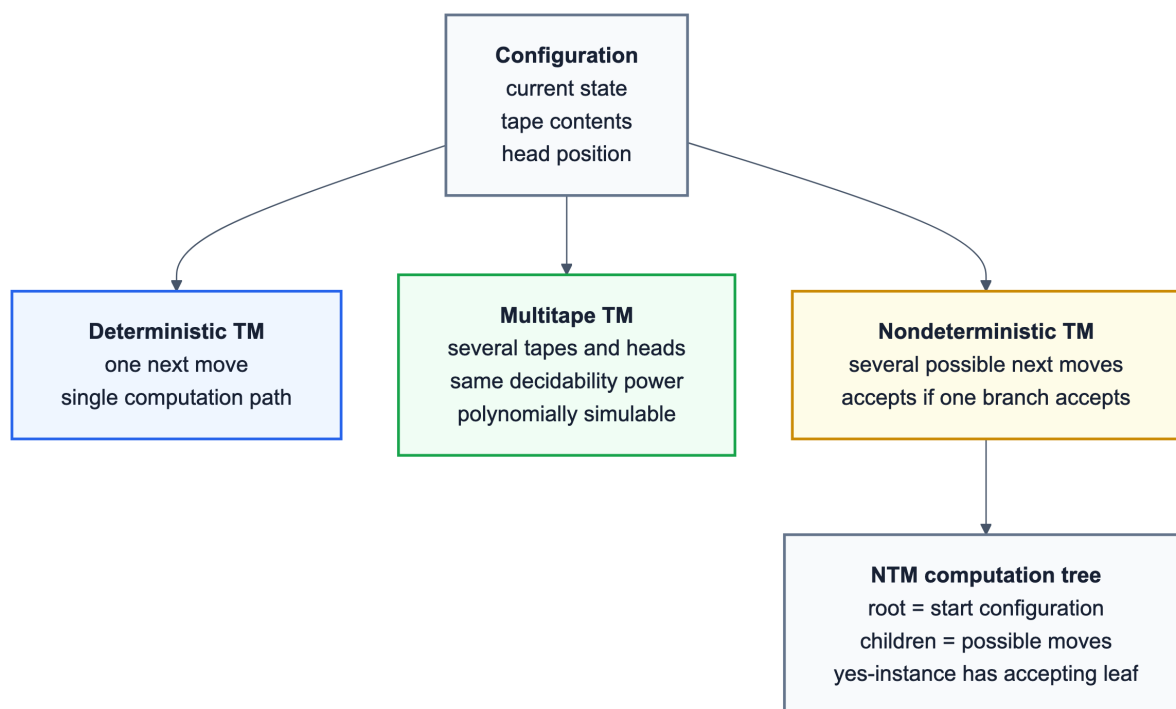


Figure 4: Turing-machine models: configurations lead to deterministic, multitape, and nondeterministic machine views.

- “Guess” a certificate and verify it deterministically.
- Existential computation: accept iff there exists an accepting branch.

For time complexity, an NTM runs in time $T(n)$ if every branch halts within $T(n)$ steps.

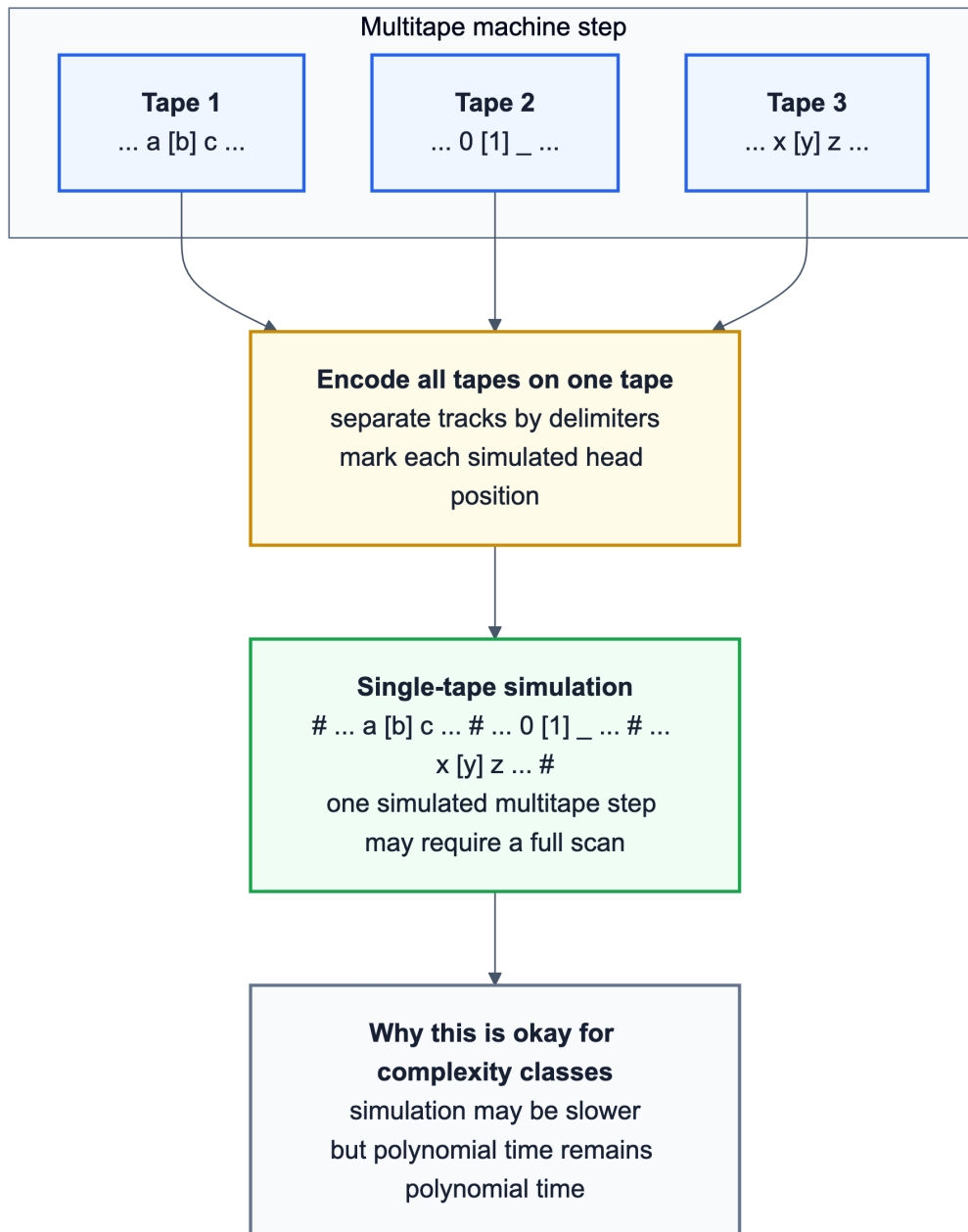


Figure 5: Multitape simulation: several tapes can be encoded on one tape using delimiters and marked head positions.

3. Decision Problems, Languages, P, NP, co-NP, Reductions, NPC

Decision problems and languages

A decision problem has yes/no answers. Once instances are encoded as strings over alphabet Σ , a decision problem corresponds to a language

$$L \subseteq \Sigma^*,$$

where $x \in L$ means instance x is a yes-instance.

Optimization problems are often studied via decision versions. Example:

- Optimization TSP: find shortest tour.
- Decision TSP: is there a tour of length at most K ?

Class P

$$\mathbf{P} = \bigcup_{k \geq 1} \text{TIME}(n^k).$$

P is the class of languages decidable by a deterministic TM in polynomial time. It is the usual theoretical notion of efficiently solvable decision problems.

Examples: graph reachability, shortest path with nonnegative weights, MST decision version, bipartite matching, linear programming.

Class NP

$$\mathbf{NP} = \bigcup_{k \geq 1} \text{NTIME}(n^k).$$

Equivalently, $L \in \mathbf{NP}$ iff there exists a polynomial-time deterministic verifier V and polynomial p such that

$$x \in L \iff \exists c, |c| \leq p(|x|) \text{ and } V(x, c) = 1.$$

The string c is a certificate/witness.

Examples: SAT, 3-SAT, CLIQUE, VERTEX-COVER, HAMILTONIAN-CYCLE, TSP decision version.

Clearly $\mathbf{P} \subseteq \mathbf{NP}$. Whether $\mathbf{P} = \mathbf{NP}$ is open.

Simple explanation

P means “we can find the answer efficiently.” NP means “if someone gives us a proposed yes-answer certificate, we can check it efficiently.” Sudoku is a good intuition: solving may be hard, but checking a filled grid is easy.

Class co-NP

For language L , its complement is $\bar{L} = \Sigma^* \setminus L$.

$$\mathbf{coNP} = \{L \mid \bar{L} \in \mathbf{NP}\}.$$

Problems in co-NP have efficiently verifiable no-certificates. Example: TAUT is in co-NP because its complement is SAT-like falsifiability.

We know $\mathbf{P} \subseteq \mathbf{NP} \cap \mathbf{coNP}$. It is unknown whether $\mathbf{NP} = \mathbf{coNP}$.

Polynomial many-one reductions

For languages A, B , a polynomial-time many-one reduction $A \leq_p B$ is a polynomial-time computable function f such that

$$x \in A \iff f(x) \in B.$$

Interpretation: if we can solve B efficiently, then we can solve A efficiently by transforming instances.

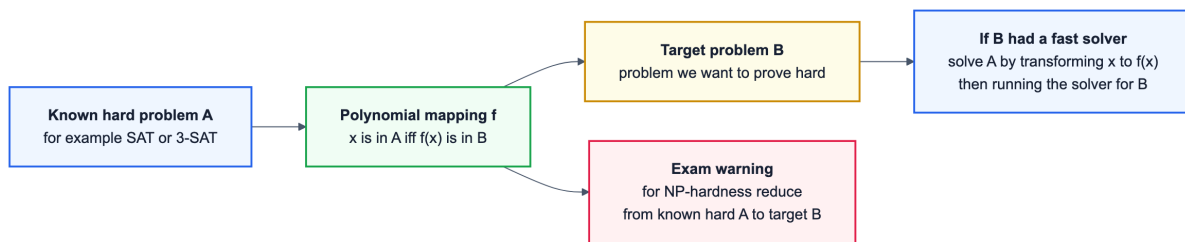


Figure 6: Polynomial reduction direction: reduce from a known hard problem to the target problem.

Use reductions to prove hardness:

1. Start from a known hard problem A .
2. Construct f mapping instances of A to instances of B .
3. Prove yes-instances map to yes-instances and no-instances to no-instances.
4. Conclude B is at least as hard as A .

Example

To prove CLIQUE reduces to INDEPENDENT-SET, map graph G to its complement $\bar{G}$ and keep the same k . A k -clique in G is exactly a k -independent set in $\bar{G}$, because edges become non-edges and non-edges become edges.

NP-hard and NP-complete

A language B is **NP-hard** if every $A \in \mathbf{NP}$ reduces to B : $A \leq_p B$.

A language B is **NP-complete** if:

1. $B \in \mathbf{NP}$,
2. B is NP-hard.

If any NP-complete problem is in P, then $\mathbf{P} = \mathbf{NP}$.

Exam tip

To prove a problem is NP-complete, show both parts: it is in NP, and it is NP-hard. For hardness, reduce from a known NP-complete problem to your problem, not the other way around.

Cook-Levin theorem

Cook-Levin theorem: SAT is NP-complete.

Proof idea:

1. SAT is in NP: a truth assignment is a certificate, and formula evaluation is polynomial.
2. For any $L \in \mathbf{NP}$, take a polynomial-time NTM M deciding L .
3. Encode an accepting computation tableau of M on input x as a Boolean formula.
4. The formula is satisfiable iff M has an accepting computation branch on x .

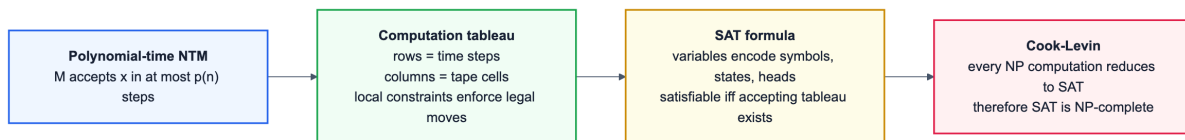


Figure 7: Cook-Levin tableau: a polynomial-time nondeterministic computation is encoded as a SAT formula.

This provides the first NP-complete problem. Many other NP-completeness proofs reduce from SAT or 3-SAT.

Heuristics and approximation for NP-complete problems

For NP-hard optimization problems, exact polynomial-time algorithms are unlikely. Practical approaches:

- **Branch and bound:** exact, but exponential worst case; uses bounds to prune search.
- **Local search:** iteratively improve a candidate solution, e.g. k-OPT for TSP.
- **Greedy heuristics:** fast, sometimes with guarantees.
- **Randomized/metaheuristics:** simulated annealing, genetic algorithms, tabu search.
- **Approximation algorithms:** polynomial-time algorithms with provable ratio.

For minimization, algorithm A has approximation ratio $\rho \geq 1$ if

$$A(I) \leq \rho \cdot \text{OPT}(I)$$

for every instance I . For maximization, use $A(I) \geq \text{OPT}(I)/\rho$.

Some problems have approximation schemes:

- PTAS: for every fixed $\epsilon > 0$, gives $(1 + \epsilon)$ -approximation in polynomial time.
- FPTAS: polynomial in both input size and $1/\epsilon$.

Example

For metric TSP, a 2-approximation means the produced tour is never more than twice the optimal tour length. If $OPT = 100$, the algorithm promises a tour of length at most 200, although it may do better on a particular instance.

4. Space Complexity, PSPACE, NPSPACE, Savitch Theorem

Space classes

For a function $s(n)$:

- $\text{DSPACE}(s(n))$: languages decidable by a deterministic TM using $O(s(n))$ space.
- $\text{NSPACE}(s(n))$: languages decidable by a nondeterministic TM using $O(s(n))$ space.

Polynomial space:

$$\mathbf{PSPACE} = \bigcup_{k \geq 1} \text{DSPACE}(n^k),$$

$$\mathbf{NPSPACE} = \bigcup_{k \geq 1} \text{NSPACE}(n^k).$$

Basic inclusions:

$$\mathbf{P} \subseteq \mathbf{NP} \subseteq \mathbf{PSPACE}.$$

The inclusion $\mathbf{NP} \subseteq \mathbf{PSPACE}$ holds because a polynomial-time nondeterministic computation tree can be searched depth-first using only polynomial space.

Configuration graph view

For a machine using $s(n)$ space, a configuration contains state, head positions, and tape contents. The number of configurations is at most exponential in $s(n)$:

$$N = 2^{O(s(n))}.$$

Acceptance becomes reachability in this configuration graph.

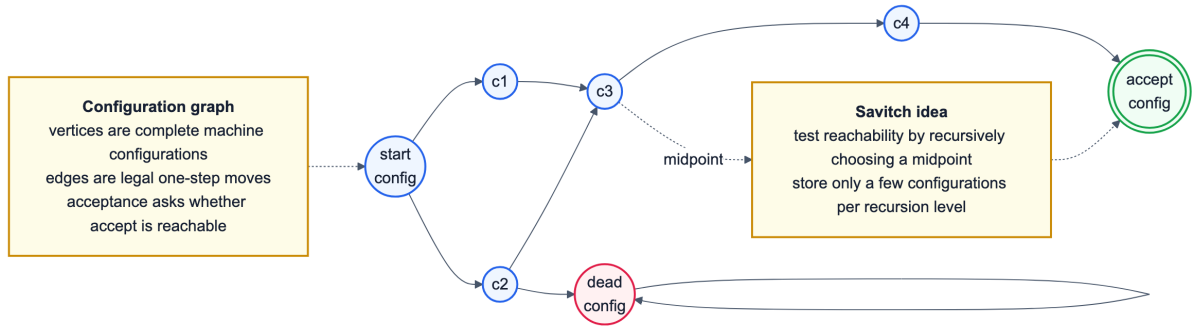


Figure 8: Configuration graph: accepting a space-bounded computation becomes a graph reachability question.

Savitch theorem

For $s(n) \geq \log n$:

$$\text{NSPACE}(s(n)) \subseteq \text{DSPACE}(s(n)^2).$$

Therefore:

$$\mathbf{PSPACE} = \mathbf{NPSPACE}.$$

Proof idea: decide reachability between configurations u and v within at most t steps by recursively guessing a midpoint m :

$$\text{Reach}(u, v, t) = \exists m : \text{Reach}(u, m, \lceil t/2 \rceil) \wedge \text{Reach}(m, v, \lfloor t/2 \rfloor).$$

The recursion depth is $O(\log t)$, and each level stores a configuration of size $O(s(n))$. Since t can be exponential in $s(n)$, $\log t = O(s(n))$, yielding $O(s(n)^2)$ deterministic space.

Simple explanation

Savitch's theorem says nondeterminism saves much less space than it seems. Instead of storing the whole nondeterministic search tree, a deterministic machine recursively asks: "is there some middle configuration that connects the start to the goal?"

5. Randomized Algorithms and Randomized Complexity Classes

Randomized algorithms

A randomized algorithm has access to random bits. Its output or running time is a random variable.

Two important types:

- **Monte Carlo:** bounded running time, may return wrong answer with small probability.
- **Las Vegas:** always correct, running time is random; expected running time is bounded.

Randomization is useful for symmetry breaking, hashing, sampling, primality testing, randomized quicksort, and randomized reductions.

Randomized Turing machine

A randomized TM can be modeled as a deterministic TM with an extra tape of random bits, or as a machine with probabilistic transitions. Acceptance probability is taken over random choices.

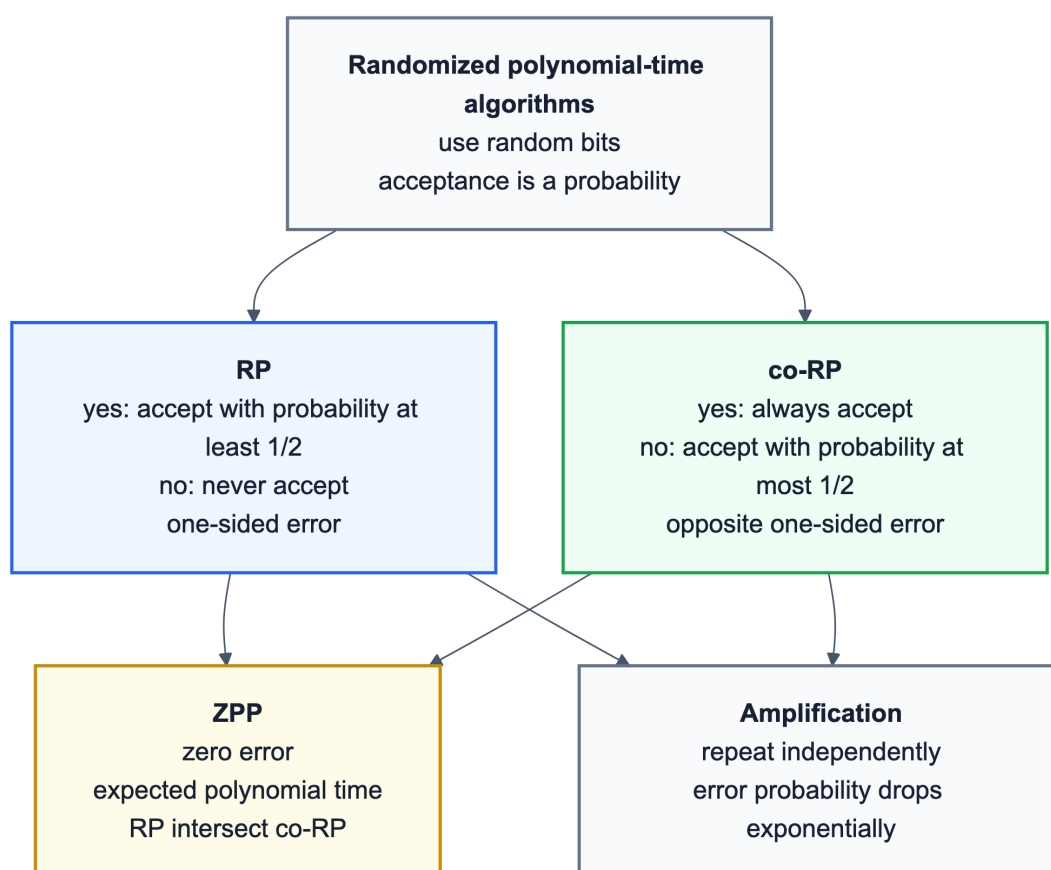


Figure 9: Randomized classes: RP and co-RP are one-sided-error classes, and ZPP is their zero-error intersection.

RP

A language L is in RP if there is a polynomial-time randomized algorithm A such that:

$$x \in L \Rightarrow \Pr[A(x) = 1] \geq \frac{1}{2},$$

$$x \notin L \Rightarrow \Pr[A(x) = 1] = 0.$$

RP has one-sided error: it never falsely accepts no-instances, but may falsely reject yes-instances. Repeating independently reduces error exponentially.

co-RP

$L \in \mathbf{coRP}$ iff $\bar{L} \in \mathbf{RP}$. Equivalently, there is a polynomial-time randomized algorithm with:

$$x \in L \Rightarrow \Pr[A(x) = 1] = 1,$$

$$x \notin L \Rightarrow \Pr[A(x) = 1] \leq \frac{1}{2}.$$

It never falsely rejects yes-instances, but may falsely accept no-instances.

ZPP

ZPP is zero-error expected polynomial time. Equivalent characterizations:

$$\mathbf{ZPP} = \mathbf{RP} \cap \mathbf{coRP}.$$

Another view: algorithm outputs correct answer or “I do not know”; probability of “I do not know” is bounded away from 1, and repetition gives expected polynomial time.

Basic relationships:

$$\mathbf{P} \subseteq \mathbf{ZPP} \subseteq \mathbf{RP} \subseteq \mathbf{NP},$$

and

$$\mathbf{P} \subseteq \mathbf{ZPP} \subseteq \mathbf{coRP} \subseteq \mathbf{coNP}.$$

Related class: BPP

BPP is not explicitly required in the exam outline, but it appears in one of the source notes and is useful context. It is the class of languages decidable by a polynomial-time randomized algorithm with bounded two-sided error:

$$\Pr[A(x) = \chi_L(x)] \geq \frac{2}{3}.$$

The constant $2/3$ is arbitrary; any constant greater than $1/2$ can be amplified by independent repetition and majority vote. RP and co-RP are one-sided-error subclasses of BPP:

$$\mathbf{RP} \subseteq \mathbf{BPP}, \quad \mathbf{coRP} \subseteq \mathbf{BPP}.$$

6. Decidability and Undecidability

Recursive and recursively enumerable languages

A language L is **recursive** or **decidable** if some TM halts on every input and accepts exactly the strings in L .

A language L is **recursively enumerable (RE)** or **Turing-recognizable** if some TM accepts exactly strings in L ; on strings outside L it may reject or loop forever.

Relationships:

- Every decidable language is RE.
- L is decidable iff both L and $\overline{L}$ are RE.
- Some RE languages are undecidable.
- Some languages are not RE.

Simple explanation

Decidable means the machine always eventually answers yes or no. Recognizable means the machine eventually says yes for true cases, but for false cases it may run forever. That one-sided behavior is the whole source of many undecidability results.

Universal language and universal Turing machine

Encode a TM M and input w as a string $\langle M, w \rangle$.

The universal language is

$$L_U = \{\langle M, w \rangle \mid M \text{ accepts } w\}.$$

A universal Turing machine simulates M on w . Thus L_U is RE: simulate $M(w)$ and accept if it accepts.

However, L_U is not decidable. If it were decidable, the halting/acceptance problem could be solved, contradicting diagonalization.

Diagonal language

Let $M_1, M_2, \dots$ be an enumeration of Turing machines. Define the diagonal language

$$L_D = \{w_i \mid M_i \text{ does not accept } w_i\},$$

where w_i is the i -th string in a fixed enumeration.

If some TM decided L_D , then considering its own index yields contradiction: it would accept its own encoding iff it does not accept it.

This diagonal argument proves that there are undecidable languages and that some languages are not recursively enumerable.

Halting problem

The halting problem asks:

$$HALT = \{\langle M, w \rangle \mid M \text{ halts on } w\}.$$

It is undecidable. Classic proof by contradiction:

1. Suppose decider $H(M, w)$ says whether M halts on w .
2. Build machine $D(x)$: if $H(x, x)$ says “halts”, loop forever; otherwise halt.
3. Run $D(D)$. If it halts, it must loop; if it loops, it must halt. Contradiction.

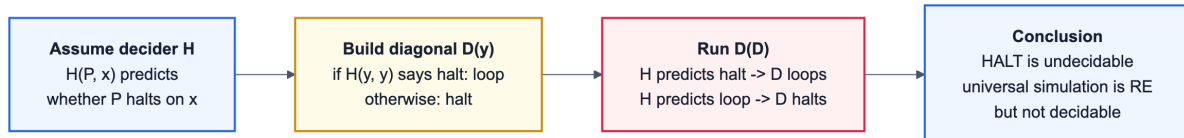


Figure 10: Diagonalization for the halting problem: assuming a decider lets us build a program that contradicts its own prediction.

Example

Think of a hypothetical program `halts(program, input)`. The diagonal trick builds a new program that does the opposite of what `halts` predicts on itself. If `halts` says the new program halts, it loops; if `halts` says it loops, it halts. So `halts` cannot exist.

Acceptance problem

The acceptance problem

$$A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts } w\}$$

is RE but undecidable. It is RE because a universal TM can simulate and accept when M accepts. It is undecidable by reduction from the halting problem or by direct diagonalization.

Proving undecidability by reduction

To prove problem B undecidable:

1. Start from known undecidable problem A .
2. Construct a computable mapping from instances of A to instances of B .
3. Show that solving B would solve A .
4. Since A is undecidable, B must be undecidable.

Common source problems: $HALT$, A_{TM} , emptiness/equivalence variants for TMs, Post correspondence problem.

Decidable vs. semi-decidable behavior

For an RE but undecidable language, there is an algorithm that eventually confirms yes-instances, but no algorithm that always answers both yes and no.

For a co-RE but not decidable language, no-instances can be confirmed.

If neither L nor $\bar{L}$ is RE, there is no recognizer for either side.

Exam Checklist

Be able to:

- State and use $O, \Omega, \Theta, o, \omega$.
- Prove total correctness via invariant plus variant.
- Define DTM, multitape TM, and NTM, and explain why multitape TMs do not change polynomial classes.
- Translate decision problems to languages.
- Define P, NP, co-NP, NP-hard, NP-complete, and polynomial reduction.
- State Cook-Levin theorem and its computation-tableau idea.
- Explain exact, heuristic, and approximation approaches for NP-hard problems.
- Define PSPACE and NPSPACE and state Savitch theorem.
- Define RP, co-RP, and ZPP, including one-sided vs. zero error.
- Distinguish decidable, RE, and non-RE languages.
- Explain diagonalization, universal language, universal TM, and the halting problem.

Common Part 3: KO

Combinatorial optimization problems: formulation, complexity analysis, algorithms, and example applications.

Primary source repositories consulted: mohwald/oi-mszz, pan-sveta/oi-si-statnice, Pryx/oi-si-mszz, dralii/oi-mszz. The final text below is synthesized and corrected against standard combinatorial optimization material.

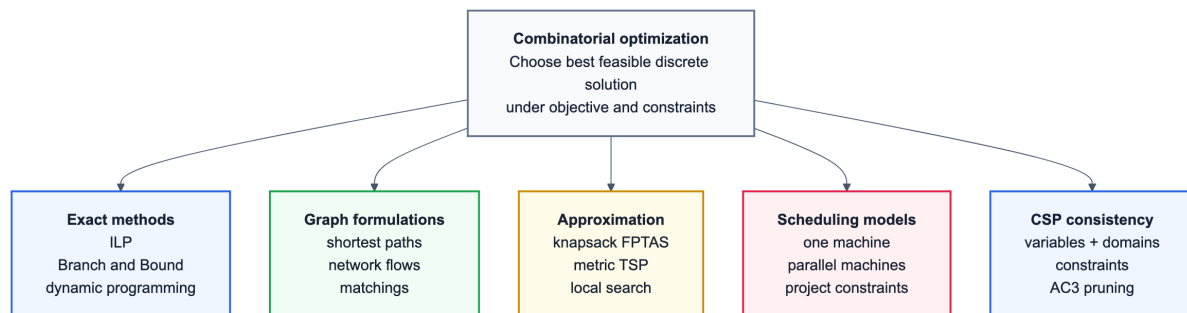


Figure 11: Combinatorial-optimization method map: exact methods, graph formulations, approximation, scheduling, and CSP consistency.

KO in Plain English

KO is about turning discrete choices into optimization problems. For each topic, know the model, whether it is easy or hard, and the standard exact, approximate, or graph algorithm.

1. Model the decision.

Variables, constraints, objective.

2. Pick the method.

Exact, polynomial graph algorithm, approximation, or DP.

3. State the guarantee.

Optimal, feasible, pseudo-polynomial, or approximate.

ILP and Branch & Bound

Use variables to describe choices.

- LP is polynomial; general ILP is NP-hard.
- Binary variables are yes/no switches.
- Branch & Bound splits cases and prunes by bounds.

Shortest paths and flows

Polynomial graph problems with strong structure.

- Dijkstra: nonnegative edges.
- Bellman-Ford: negative edges, detects negative cycles.
- Max-flow min-cut links augmenting paths to cuts.

Knapsack and TSP

Hard problems where DP and approximation matter.

- Knapsack DP is pseudo-polynomial.
- FPTAS scales profits for approximation.
- Metric TSP has double-tree and Christofides.

Scheduling

Assign jobs to time/resources.

- Use Graham notation to describe the problem.
- Know Bratley, Horn, list scheduling, McNaughton.
- ILP models use start/order/time-indexed variables.

CSP and AC3

Reduce variable domains by consistency.

- CSP = variables, domains, constraints.
- Arc consistency removes unsupported values.
- AC3 repeats revisions until stable or empty domain.

How to answer

Start with the model; then discuss algorithmic status.

- What are the variables?
- What is optimized?
- Is the method exact, approximate, or heuristic?

Exam sentence: ILP is powerful because it models discrete choices with linear constraints, but integrality makes it NP-hard; Branch & Bound solves it by relaxing, branching, and pruning.

Three traps

- Pseudo-polynomial does not mean polynomial in input length.
- Dijkstra needs nonnegative edge weights.
- Approximation ratios depend on minimization vs. maximization.

Use this page to choose the right method before reading formulas.

1. Integer Linear Programming and Branch and Bound

Linear and integer linear programs

A linear program (LP) optimizes a linear objective under linear constraints:

$$\min c^T x \quad \text{s.t.} \quad Ax \leq b, \quad x \geq 0.$$

An integer linear program (ILP) additionally requires some or all variables to be integer:

$$x_i \in \mathbb{Z}$$

or, in binary ILP,

$$x_i \in \{0, 1\}.$$

LP is solvable in polynomial time. General ILP is NP-hard, even with binary variables.

Simple explanation

ILP is a modeling language for “choose discrete decisions under linear rules.” Binary variables are yes/no switches. The hard part is not writing the model; it is that requiring variables to be integer destroys the nice geometry that makes ordinary LP polynomial-time solvable.

Modeling with ILP

Typical modeling choices:

- Binary decision variable: $x_i = 1$ iff item/action/edge/job is selected.
- Assignment variable: $x_{ij} = 1$ iff object i is assigned to option j .
- Ordering variable: $y_{ij} = 1$ iff i precedes j .
- Flow variable: f_e is amount sent through edge e .

Common constraints:

- Select exactly one: $\sum_j x_{ij} = 1$.
- Capacity: $\sum_i a_i x_i \leq C$.
- Logical implication: $x \leq y$ means $x = 1 \Rightarrow y = 1$.
- Either-or via big-M: $a^T x \leq b + M(1 - y)$.

Shortest path ILP formulation

For directed graph $G = (V, E)$ with source s , target t , edge costs c_e , let $x_e \in \{0, 1\}$ indicate whether edge e is selected.

$$\min \sum_{e \in E} c_e x_e$$

subject to flow conservation:

$$\sum_{e \in \delta^+(v)} x_e - \sum_{e \in \delta^-(v)} x_e = \begin{cases} 1 & v = s, \\ -1 & v = t, \\ 0 & \text{otherwise.} \end{cases}$$

For shortest path, the LP relaxation $0 \leq x_e \leq 1$ already has integral optimal solutions because the node-arc incidence matrix is totally unimodular.

Traveling salesman ILP formulation

For complete graph $G = (V, E)$ with costs c_{ij} , binary variable $x_{ij} = 1$ iff edge $\{i, j\}$ is in the tour.

Degree constraints:

$$\sum_{j \neq i} x_{ij} = 2 \quad \forall i \in V.$$

Subtour elimination constraints:

$$\sum_{\{i,j\} \in E(S)} x_{ij} \leq |S| - 1 \quad \forall S \subset V, 2 \leq |S| \leq |V| - 1.$$

Objective:

$$\min \sum_{\{i,j\} \in E} c_{ij} x_{ij}.$$

This Dantzig-Fulkerson-Johnson formulation has exponentially many subtour constraints, but they can be separated iteratively. Alternative compact formulations include MTZ constraints with order variables, usually weaker but smaller.

Branch and Bound

Branch and Bound is an exact tree-search framework for hard optimization problems.

For minimization:

1. Solve a relaxation of the current subproblem to get lower bound LB .
2. If relaxation is infeasible, prune.
3. If $LB \geq UB$ where UB is best known feasible solution, prune.
4. If relaxation solution is integral/feasible for original problem, update UB .
5. Otherwise branch, e.g. choose fractional variable x_i and create $x_i \leq \lfloor x_i^* \rfloor$ and $x_i \geq \lceil x_i^* \rceil$.

Worst-case complexity remains exponential, but good bounds and branching rules make many practical instances solvable.

Other exact ILP methods

For completeness, the source notes also mention the standard family of exact ILP methods:

- **Enumerative methods:** try feasible integer assignments, often only practical for tiny instances.
- **Cutting planes:** solve the LP relaxation, add valid inequalities that cut off the current fractional solution, and repeat.
- **Branch and cut:** combine branch-and-bound with dynamically generated cutting planes. This is the basis of many modern ILP solvers.

Special ILP problems solvable in polynomial time

Some ILPs have integral LP relaxations. Important sufficient condition: the constraint matrix is **totally unimodular** and the right-hand side is integral.

Examples:

- Shortest path and many network-flow formulations.
- Maximum flow and minimum-cost flow.
- Bipartite matching.
- Assignment problem.
- Certain interval/scheduling matrices.

In these cases, solving the LP gives an integral optimum.

2. Shortest Paths

Problem variants

Given graph $G = (V, E)$ and edge lengths $c : E \rightarrow \mathbb{R}$:

- Single-source shortest paths: distances from s to all vertices.
- Single-pair shortest path: distance from s to t .
- All-pairs shortest paths: distances between all pairs.

If a negative cycle is reachable from s and can reach t , no finite shortest path from s to t exists.

Bellman's principle of optimality: every subpath of a shortest path is itself shortest between its endpoints.

Relaxation

For edge (u, v) , relaxation checks whether going through u improves v :

$$\text{if } d[v] > d[u] + c(u, v), \text{ set } d[v] = d[u] + c(u, v).$$

Store predecessor pointers to reconstruct paths.

Example

If currently $d[A] = 3$ and edge (A, B) has cost 4, then the path to B through A has cost 7. If $d[B] = 10$, relaxation updates $d[B]$ to 7 and stores predecessor A . If $d[B] = 6$, nothing changes.

Dijkstra algorithm

Assumption: all edge lengths are nonnegative.

1. Initialize $d[s] = 0$, all others ∞ .
2. Maintain priority queue keyed by tentative distance.
3. Repeatedly extract vertex u with minimum distance.
4. Relax all outgoing edges (u, v) .

Once a vertex is extracted, its distance is final. Complexity with binary heap: $O((n + m) \log n)$, usually written $O(m \log n)$. With Fibonacci heap: $O(m + n \log n)$.

Applications: routing with nonnegative costs, road networks, uniform-cost search.

Bellman-Ford algorithm

Allows negative edges and detects reachable negative cycles.

1. Initialize distances.
2. Repeat $n - 1$ times: relax every edge.
3. Do one extra pass; if any edge can still be relaxed, a negative cycle is reachable.

Complexity: $O(nm)$.

Applications: difference constraints, graphs with negative penalties, arbitrage detection.

Floyd-Warshall algorithm

All-pairs shortest paths by dynamic programming. Let $D^{(k)}[i, j]$ be the shortest distance from i to j using only intermediate vertices from $\{1, \dots, k\}$.

Recurrence:

$$D^{(k)}[i, j] = \min(D^{(k-1)}[i, j], D^{(k-1)}[i, k] + D^{(k-1)}[k, j]).$$

Implementation:

```
for k in 1..n:
  for i in 1..n:
    for j in 1..n:
      D[i][j] = min(D[i][j], D[i][k] + D[k][j])
```

Complexity: $O(n^3)$ time, $O(n^2)$ memory. Negative cycle exists iff some $D[i, i] < 0$ after the algorithm.

Shortest paths in DAGs

In a directed acyclic graph:

1. Topologically sort vertices.
2. Initialize $d[s] = 0$.
3. Process vertices in topological order and relax outgoing edges.

Complexity: $O(n + m)$, works with negative edges because there are no cycles.

Problem formulations using shortest paths

Shortest-path models appear in:

- Difference constraints: $x_v - x_u \leq c(u, v)$.
- Project scheduling with precedence constraints.
- Dynamic programming state graphs.
- Resource-constrained path variants.
- Viterbi-style decoding in layered graphs.

3. Network Flows

Maximum flow

For directed graph $G = (V, E)$, source s , sink t , capacities $u_e \geq 0$, a flow f satisfies:

Capacity:

$$0 \leq f_e \leq u_e.$$

Flow conservation for $v \neq s, t$:

$$\sum_{e \in \delta^-(v)} f_e = \sum_{e \in \delta^+(v)} f_e.$$

Flow value:

$$|f| = \sum_{e \in \delta^+(s)} f_e - \sum_{e \in \delta^-(s)} f_e.$$

Goal: maximize $|f|$.

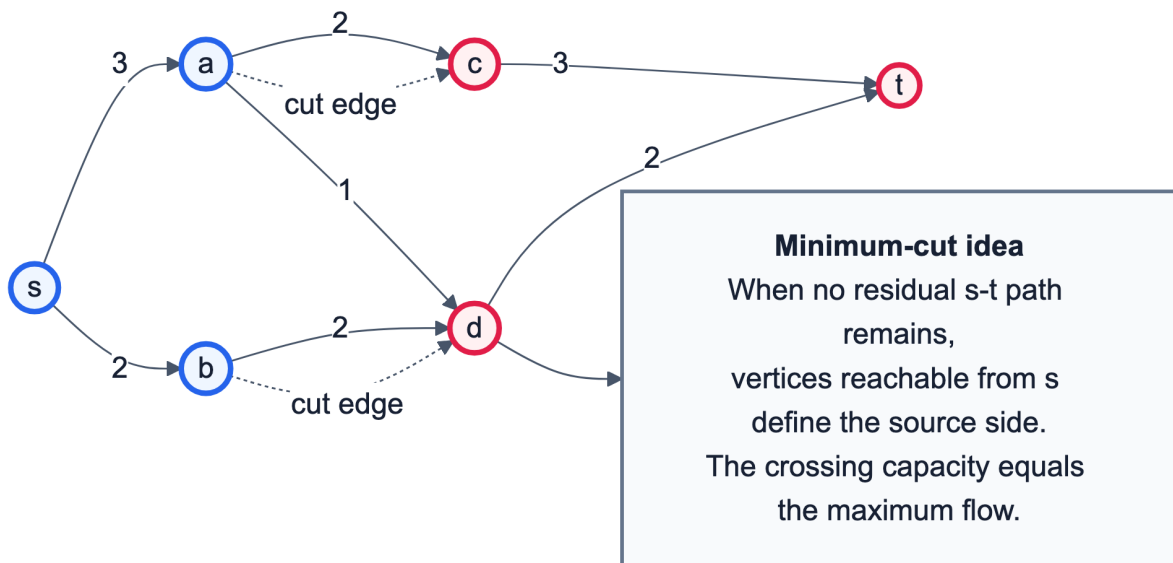


Figure 12: Maximum-flow/minimum-cut picture: augmenting paths improve flow until the residual reachable set defines a minimum cut.

Residual graph

For each edge (u, v) :

- Forward residual capacity: $u(u, v) - f(u, v)$.
- Backward residual capacity: $f(u, v)$.

An augmenting path is an s - t path in the residual graph with positive residual capacity. Its bottleneck is the minimum residual capacity along the path.

Example

If an augmenting path has residual capacities 5, 2, 7, the bottleneck is 2. Increase flow by 2 along every forward edge on the path. Any backward edge means “undo 2 units” of previous flow on that original edge.

Ford-Fulkerson algorithm

1. Start with zero flow.
2. While an augmenting path exists, augment by its bottleneck capacity.
3. Stop when no augmenting path exists.

With integral capacities, terminates with maximum flow. Complexity depends on path choice; with arbitrary real capacities it may not terminate.

Edmonds-Karp chooses shortest augmenting paths by BFS and runs in $O(nm^2)$.

Minimum cut and max-flow min-cut theorem

An s - t cut is a partition $(S, V \setminus S)$ with $s \in S$, $t \notin S$. Its capacity is

$$c(S, V \setminus S) = \sum_{u \in S, v \notin S} u(u, v).$$

Max-flow min-cut theorem:

$$\max |f| = \min c(S, V \setminus S).$$

At the end of Ford-Fulkerson, vertices reachable from s in the residual graph define a minimum cut.

Simple explanation

The residual graph shows where more flow can still be pushed or undone. When the sink is no longer reachable in this graph, the reachable vertices form the source side of a bottleneck cut. That is why “no augmenting path” proves optimality.

Feasible flow with balances

General circulation with balances has node demands/supplies $b(v)$:

$$\sum_{e \in \delta^-(v)} f_e - \sum_{e \in \delta^+(v)} f_e = b(v).$$

Convention: $b(v) > 0$ means demand, $b(v) < 0$ means supply. Feasibility requires $\sum_v b(v) = 0$.

With lower and upper bounds $l_e \leq f_e \leq u_e$:

1. Substitute $f'_e = f_e - l_e$ so $0 \leq f'_e \leq u_e - l_e$.
2. Adjust balances:

$$b'(v) = b(v) - \sum_{e \in \delta^-(v)} l_e + \sum_{e \in \delta^+(v)} l_e.$$

3. Add super-source to supply nodes and super-sink from demand nodes, or use equivalent circulation construction.
4. Feasible iff all artificial demand/supply edges can be saturated.

Minimum-cost flow

Given capacities, balances, and costs c_e , minimize

$$\sum_{e \in E} c_e f_e$$

subject to flow constraints.

Minimum-cost flow generalizes shortest path, assignment, transportation, and some scheduling problems.

Cycle-canceling algorithm

1. Start with any feasible flow.
2. Build residual graph with residual costs: forward edge cost c_e , backward edge cost $-c_e$.
3. If the residual graph contains a negative-cost cycle, augment flow around it as much as possible.
4. Repeat until no negative cycle remains.

Optimality condition: a feasible flow is minimum-cost iff its residual graph has no negative-cost cycle.

Maximum cardinality matching via flow

For bipartite graph $G = (U \cup W, E)$:

1. Add source s and sink t .
2. Add edges $s \rightarrow u$ for $u \in U$, capacity 1.
3. Direct original edges $u \rightarrow w$ with capacity 1.
4. Add edges $w \rightarrow t$ for $w \in W$, capacity 1.

Maximum integral flow corresponds to maximum cardinality matching. Integrality follows from integral capacities.

Problem formulations using flows

Flows model:

- Assignment and bipartite matching.
- Transportation and logistics.
- Image segmentation by min cut.
- Project selection.
- Disjoint paths and connectivity.
- Scheduling with time-expanded networks.

4. Knapsack

0/1 knapsack

Input: items $i = 1, \dots, n$ with weight $w_i > 0$, profit $p_i > 0$, capacity W .

$$\max \sum_i p_i x_i$$

subject to:

$$\sum_i w_i x_i \leq W, \quad x_i \in \{0, 1\}.$$

0/1 knapsack is weakly NP-hard.

Fractional knapsack

If fractions are allowed, greedy by decreasing ratio p_i/w_i is optimal. Complexity is dominated by sorting: $O(n \log n)$.

Dynamic programming by weight

Let $DP[i][w]$ be maximum profit using first i items with capacity w .

$$DP[i][w] = \begin{cases} DP[i-1][w], & w_i > w, \\ \max(DP[i-1][w], DP[i-1][w-w_i] + p_i), & w_i \leq w. \end{cases}$$

Complexity: $O(nW)$ time and $O(W)$ memory with rolling array. This is pseudo-polynomial, not polynomial in input bit-length.

Dynamic programming by profit

Let $DP[i][p]$ be minimum weight needed to achieve profit exactly p using first i items. Minimize weight and find largest profit with weight $\leq W$.

Complexity: $O(nP)$ where $P = \sum_i p_i$.

Approximation and FPTAS

A simple 2-approximation:

1. Take greedy items by profit/weight ratio until next item does not fit.
2. Compare this set with the single most profitable item that fits.
3. Return the better one.

This achieves at least $OPT/2$.

FPTAS by profit scaling:

1. Choose scaling factor $K = \epsilon p_{\max}/n$.
2. Replace profits by $p'_i = \lfloor p_i/K \rfloor$.
3. Run profit-DP on scaled profits.

Runs polynomially in n and $1/\epsilon$, and returns a $(1 - \epsilon)$ -approximation for maximization.

5. Traveling Salesman Problem

Problem statement and complexity

TSP asks for a minimum-cost Hamiltonian cycle visiting every city exactly once.

Decision version: is there a tour of length at most B ?

TSP is NP-hard. The decision version is NP-complete. General metric-free TSP has no finite approximation ratio unless $P=NP$, because missing Hamiltonian edges can be encoded as very expensive edges.

Metric TSP

Metric TSP assumes:

$$c(i, j) \geq 0, \quad c(i, j) = c(j, i), \quad c(i, j) \leq c(i, k) + c(k, j).$$

The triangle inequality enables shortcutting repeated vertices without increasing tour length.

Double-tree algorithm

1. Compute an MST T .
2. Double every edge of T ; the resulting multigraph is Eulerian.
3. Traverse an Euler tour.
4. Shortcut already visited vertices to obtain a Hamiltonian cycle.

Guarantee:

$$w(T) \leq OPT,$$

so doubled tree has weight $2w(T) \leq 2OPT$, and shortcutting does not increase weight. Approximation ratio: 2.

Christofides algorithm

1. Compute MST T .
2. Let O be the set of odd-degree vertices in T .
3. Compute a minimum-weight perfect matching M on O .
4. Add M to T ; all degrees become even.
5. Find Euler tour and shortcut repeated vertices.

Guarantee:

$$w(T) \leq OPT, \quad w(M) \leq \frac{1}{2}OPT.$$

Therefore total weight is at most $\frac{3}{2}OPT$. Approximation ratio: 1.5 for metric TSP.

Exam tip

Both double-tree and Christofides rely on the triangle inequality. Without it, shortcutting repeated vertices can make the tour more expensive, and the approximation proof collapses.

Local search and k-OPT

k-OPT improves a tour by replacing k edges with k different edges, preserving a Hamiltonian cycle.

- 2-OPT removes two crossing or bad edges and reconnects.
- 3-OPT is stronger but more expensive.
- General k-OPT grows quickly with k .

Local search stops at a local optimum: no improving k-edge exchange exists. It is usually heuristic; quality can be good in practice, but global optimality is not guaranteed.

6. Scheduling

Basic definitions

Input consists of jobs $J_1, \dots, J_n$ and resources/machines. Common job parameters:

- p_j : processing time.
- r_j : release time, earliest start.
- $\tilde{d}_j$: hard deadline.
- d_j : due date used in lateness/tardiness objectives.
- w_j : job weight.
- C_j : completion time.
- $L_j = C_j - d_j$: lateness.
- $T_j = \max(0, C_j - d_j)$: tardiness.
- $C_{\max} = \max_j C_j$: makespan.

Preemption (**pmtn**) means a job can be interrupted and resumed later.

Graham notation

Scheduling problems are denoted:

$$\alpha \mid \beta \mid \gamma.$$

- α : machine environment. Examples: 1 one machine, P identical parallel machines, Q uniform parallel machines, R unrelated machines.
- β : job constraints. Examples: **r_j**, **d_j**, **pmtn**, **prec**.
- γ : objective. Examples: $C_{\max}$, $\sum C_j$, $\sum w_j C_j$, $L_{\max}$, $\sum T_j$.

Simple explanation

Read Graham notation as “machine environment | job restrictions | objective.” For example, $P|prec|C_{\max}$ means identical parallel machines, precedence constraints between jobs, and the goal is to minimize the finish time of the last job.

One resource: Bratley algorithm

Bratley’s algorithm is a branch-and-bound algorithm for single-machine scheduling with release times and deadlines, often written:

$$1 \mid r_j, \tilde{d}_j \mid C_{\max}$$

as a feasibility/makespan problem.

It searches over job orders. At a partial schedule:

- Current time t is the completion time of scheduled jobs.
- A candidate job j starts at $\max(t, r_j)$ and completes at $C_j = \max(t, r_j) + p_j$.
- If $C_j > \tilde{d}_j$, that branch is infeasible.

Pruning rules:

- **Deadline pruning**: if any remaining job cannot meet its deadline even when scheduled next, prune.
- **Bound pruning**: if a lower bound on final makespan exceeds best known solution, prune.

- **Dominance/decomposition:** if no remaining job is released before current idle boundary, some ordering choices become irrelevant.

Worst-case exponential, but effective for moderate instances.

One resource: Horn algorithm

Horn's algorithm solves the preemptive single-machine problem:

$$1 \mid r_j, \text{pmtn} \mid L_{\max}.$$

Rule: at every time, among available jobs, process the job with earliest due date/deadline. This is the preemptive earliest-deadline-first rule.

Implementation with priority queue by due date gives $O(n \log n)$ after sorting events. It is optimal for minimizing maximum lateness with release times and preemption.

Parallel identical resources: list scheduling

For identical parallel machines and precedence constraints:

$$P \mid \text{prec} \mid C_{\max},$$

list scheduling keeps a priority list of available jobs whose predecessors are completed. Whenever a machine becomes free, assign the next available job from the list.

For m identical machines, Graham's list scheduling has approximation guarantee:

$$C_{\max}^{LS} \leq \left(2 - \frac{1}{m}\right) OPT.$$

If jobs are independent and sorted by longest processing time first (LPT), the guarantee improves to:

$$\frac{4}{3} - \frac{1}{3m}.$$

Parallel identical resources: McNaughton algorithm

For the preemptive identical-machine problem

$$P \mid \text{pmtn} \mid C_{\max},$$

McNaughton's algorithm gives an optimal schedule in $O(n)$ time after the jobs are ordered. The optimal makespan is

$$C_{\max}^* = \max \left\{ \max_j p_j, \frac{1}{m} \sum_j p_j \right\}.$$

Place the jobs one after another into an $m \times C_{\max}^*$ rectangle. When a job crosses a machine boundary, split it there; preemption makes this legal. The lower bound is therefore attainable.

Parallel identical resources: dynamic programming

For fixed number of machines m , problem

$$P_m \parallel C_{\max}$$

can be solved pseudo-polynomially by dynamic programming over machine loads. For two machines, this is closely related to PARTITION.

For constant m , maintain reachable load vectors after scheduling first i jobs. Complexity is pseudo-polynomial in total processing time and exponential in m .

Project scheduling with temporal constraints

Project scheduling has activities and temporal relations such as:

$$s_j - s_i \geq a_{ij}$$

or equivalently difference constraints. If only temporal constraints exist and resources are unlimited, feasibility and earliest times can be solved by longest paths in a constraint graph, or by Bellman-Ford after sign conversion.

With renewable resource constraints, project scheduling becomes NP-hard.

Relative-order ILP formulation Variables:

- s_i : start time of job i .
- $y_{ij} = 1$ iff job i is before job j on a shared resource.

For jobs i, j needing the same unary resource:

$$s_i + p_i \leq s_j + M(1 - y_{ij}),$$

$$s_j + p_j \leq s_i + My_{ij}.$$

Temporal constraints are linear:

$$s_j - s_i \geq a_{ij}.$$

Objective may minimize $C_{\max}$ with constraints $C_{\max} \geq s_i + p_i$.

Time-indexed ILP formulation For discrete time horizon T , let $x_{jt} = 1$ iff job j starts at time t .

Each job starts once:

$$\sum_{t=0}^{T-p_j} x_{jt} = 1.$$

Resource capacity R at each time τ :

$$\sum_j \sum_{t: t \leq \tau < t+p_j} r_{jk} x_{jt} \leq R_k.$$

Time-indexed formulations can be large but often have strong relaxations.

Example

If job j has processing time $p_j = 3$ and starts at $t = 5$, then it occupies times 5, 6, 7. In a time-indexed ILP, capacity constraints for time slots 5, 6, and 7 all include the variable $x_{j,5}$.

7. Constraint Satisfaction Problem and AC3

CSP definition

A constraint satisfaction problem consists of:

- Variables $X_1, \dots, X_n$.
- Domains D_i of possible values.
- Constraints restricting allowed combinations of values.

A solution assigns every variable a value from its domain satisfying all constraints.

Examples: graph coloring, Sudoku, scheduling, map coloring, configuration problems.

General CSP is NP-complete, but propagation and structure can make many instances easy.

Arc consistency

For binary constraint C_{ij} between variables X_i and X_j , arc (i, j) is arc-consistent iff for every value $a \in D_i$ there exists value $b \in D_j$ such that (a, b) satisfies C_{ij} .

If no such b exists, a can be removed from D_i .

AC3 algorithm

AC3 enforces arc consistency.

AC3:

```
queue = all directed arcs (Xi, Xj)
while queue not empty:
    (Xi, Xj) = pop(queue)
    if revise(Xi, Xj):
        if Di is empty: return inconsistent
        for each neighbor Xk of Xi, k != j:
            push(queue, (Xk, Xi))
return arc-consistent
```

```
revise(Xi, Xj):
    changed = false
    for each a in Di:
        if no b in Dj satisfies Cij(a,b):
            remove a from Di
            changed = true
    return changed
```

For n variables, maximum domain size d , and e binary constraints, a standard bound is $O(ed^3)$; with optimized support tracking it can be improved.

Arc consistency is not complete in general: after AC3, search/backtracking may still be needed.

Exam Checklist

Be able to:

- Formulate shortest path and TSP as ILPs.
- Explain Branch and Bound and LP relaxation bounds.
- Name ILP cases where LP relaxation is integral.
- State Dijkstra, Bellman-Ford, Floyd-Warshall, and DAG shortest paths with assumptions and complexities.
- Define residual graph, augmenting path, max-flow min-cut theorem, and Ford-Fulkerson.
- Transform feasible flow with lower bounds and balances.
- Explain minimum-cost flow and negative-cycle optimality.
- Reduce bipartite matching to max flow.
- Give knapsack DP, pseudo-polynomial meaning, 2-approximation, and FPTAS.
- Prove double-tree 2-approximation and Christofides $3/2$ -approximation for metric TSP.
- Explain Graham scheduling notation, Bratley, Horn, list scheduling, McNaughton, and DP for fixed parallel machines.
- Build relative-order and time-indexed ILP formulations for project scheduling.
- Define CSP, arc consistency, and AC3.